



RELAY

Agentic Supply-Chain Exception Resolution System

PROJECT REPORT

Project type	AI agent system — multi-agent orchestration over live operational data
Frontend	Streamlit (Python) — role-based buyer and supplier workspaces
Backend / data	Supabase (managed PostgreSQL) — 17 relational tables
Agent orchestration	LangGraph state machines, Groq-hosted LLM with a deterministic fallback
Operational data	Two public Kaggle datasets — real delivery history and inventory data
Report generated	September 2026

Table of Contents

1. Executive Summary
2. Problem Statement & Objectives
3. System Architecture
4. Technology Stack
5. Operational Data — Two Kaggle Datasets
6. Database Design
7. The Agentic Workflow — Seven Stages
8. Exception Detection & Resolution Options
9. Application Modules (Buyer & Supplier Workspaces)
10. Authentication, Roles & Governance
11. Testing Strategy
12. Results — What the System Detected
13. Challenges & Engineering Decisions
14. Limitations & Future Scope
15. Conclusion

SECTION 1

Executive Summary

RELAY is an agentic AI system that watches a company's live operational data — purchase orders, shipments and warehouse stock — and automatically detects, investigates and proposes resolutions for supply-chain exceptions: late deliveries, stock shortages, and supplier capacity bottlenecks. An AI agent gathers evidence from the real database using a set of typed tools, reasons about root cause and business impact, and drafts two to four costed resolution options. A human decision-maker reviews the evidence and approves, rejects, or overrides the recommendation before anything is written back to the operational records. Every step — every tool call, every decision, every database change — is captured in an immutable audit trail.

207

exceptions detected

6,786

shipments tracked

17

database tables

16

typed agent tools

The system is built as a full-stack, role-based web application: a public marketing site, an authentication flow backed by Supabase Auth, and two distinct operational workspaces — one for buyers (the company running the supply chain) and one for suppliers (the vendors fulfilling it). The operational data itself is not synthetic: it is loaded from two public, real-world Kaggle datasets, and every value the datasets do not carry is documented rather than invented.

SECTION 2

Problem Statement & Objectives

The problem. Supply-chain teams juggle three separate concerns — procurement (are my purchase orders on time?), logistics (are shipments moving as planned?), and inventory (do I have enough stock?) — usually across disconnected systems. When something goes wrong, a human has to manually cross-reference records to find the root cause, weigh options, and act, often under time pressure and with incomplete visibility.

The objective. Build a system that does the first 80% of that work automatically and with evidence, while keeping a human firmly in control of every action that changes real data.

Specific objectives:

- Detect exceptions automatically from live data using deterministic, explainable rules (not guesswork).
- Have an AI agent investigate each exception the way an analyst would — reading real records across procurement, logistics and inventory before drawing a conclusion.
- Generate multiple costed, ranked resolution options grounded in actual database records — never invented IDs or values.
- Enforce a human approval gate: no operational write happens until a person decides.
- Record a complete, immutable audit trail of every agent step and human decision.
- Serve two distinct roles — buyer and supplier — from one platform with role-appropriate views and permissions.

System Architecture

RELAY is organised into five layers, each with a single responsibility. Data flows down from the presentation layer to Supabase, and the agentic layer reads and writes through the same tool layer the rest of the app never touches directly.

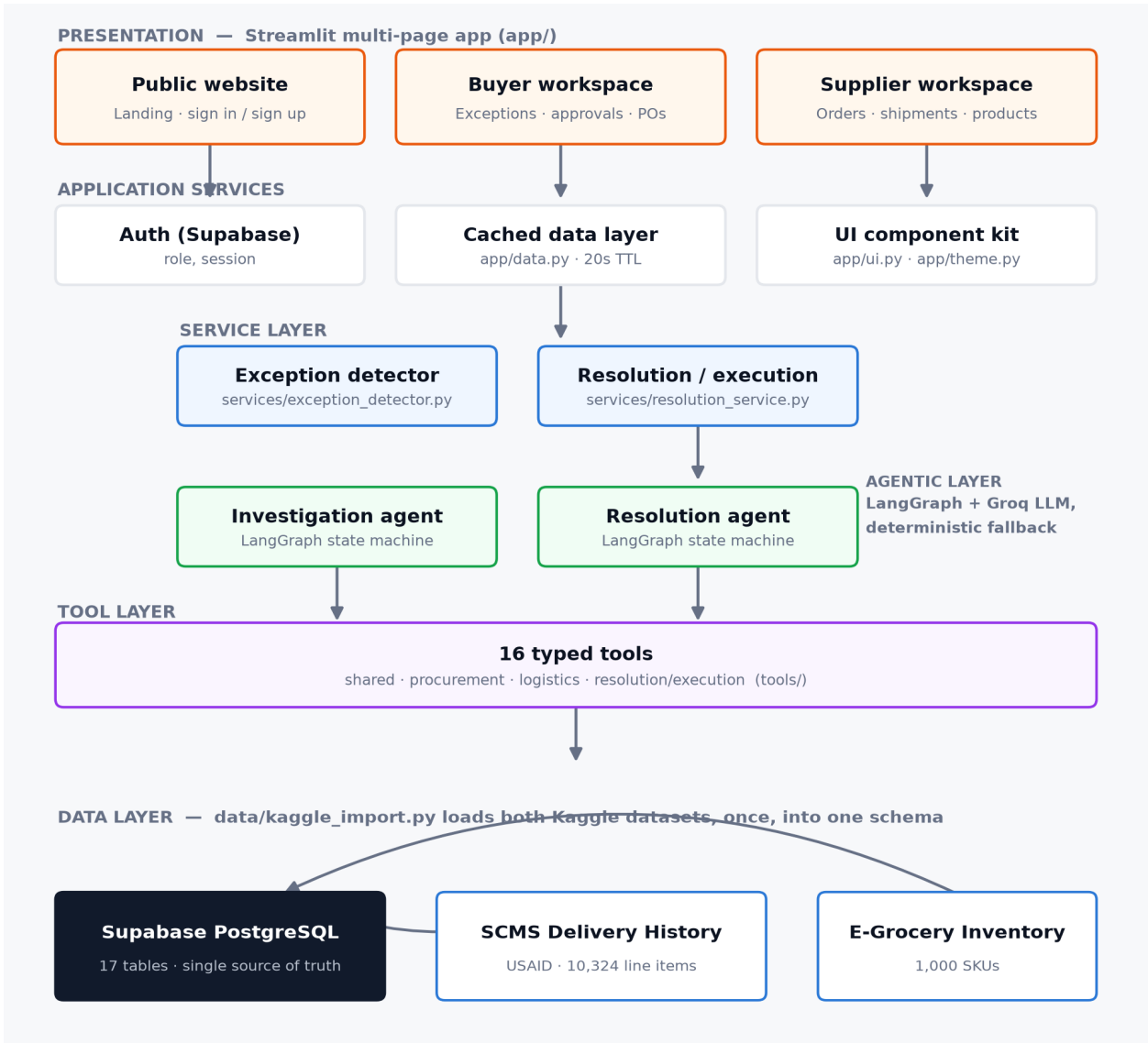


Figure 1 — Layered system architecture, from the Streamlit UI down to the two Kaggle-sourced datasets in Supabase PostgreSQL.

Layer responsibilities

Layer	Key files	Responsibility
Presentation	app/pages/*.py	Public site, auth screens, buyer workspace, supplier workspace — all Streamlit pages, styled by one shared design system (app/theme.py).
Application services	app/data.py, app/ui.py, app/shell.py	Cached reads (20-second TTL), reusable UI components (tables, badges, KPI tiles, charts), and the sidebar/navigation shell.

Layer	Key files	Responsibility
Service layer	services/*.py	Business logic: exception detection rules, investigation orchestration, resolution planning and execution, audit logging.
Agentic layer	agents/*.py	Two LangGraph state machines (investigation, resolution) that call an LLM (or a deterministic reasoner) to decide the next tool to call, then synthesise findings.
Tool layer	tools/*.py	16 typed Python functions the agent can call — each one a thin, safe wrapper over a real SQL query or write against Supabase.
Data layer	database/*.py, data/*.py	SQLAlchemy connection pool, the PostgreSQL schema, and the importer that loads the two Kaggle datasets.

Technology Stack

Every entry below is an actual dependency declared in the project's `requirements.txt`, not an aspirational choice.

Frontend

Streamlit	≥ 1.35	The entire UI — public site, auth, both workspaces — is a multi-page Streamlit app.
Plotly	≥ 5.22	All charts: status-mix bars, exposure charts, stock-position charts.
Custom CSS design system	—	<code>app/theme.py</code> — one shared token set (colours, spacing, typography) styling every Streamlit widget consistently.

Backend / Data

Supabase	≥ 2.3	Managed PostgreSQL + authentication, used as the single source of truth.
PostgreSQL	via Supabase	17 relational tables with foreign keys, CHECK constraints and indexes.
SQLAlchemy	≥ 2.0	Connection pooling and parameterised SQL execution.
psycopg2-binary	≥ 2.9	PostgreSQL driver used under SQLAlchemy.
Pydantic	≥ 2.7	Typed, validated data models for exceptions, resolutions and execution results.

AI / Agent Orchestration

LangGraph	≥ 0.2	State-machine orchestration for the investigation and resolution agents.
LangChain (core)	≥ 0.3	LLM abstraction layer used by both agents.
langchain-groq	≥ 0.2	Groq-hosted LLM inference (<code>openai/gpt-oss-120b</code>) — primary reasoning engine.
langchain-openai	≥ 0.2	OpenAI fallback provider, used only if a Groq key is not configured.

Data Engineering

kagglehub	≥ 0.3	Downloads and caches the two Kaggle datasets.
pandas	≥ 2.2	Transforms both raw CSV datasets into the relational schema.
NumPy	—	Numeric operations during the import's timeline-replay logic.

Testing & Tooling

pytest	≥ 8.2	60 tests across the data layer, agent behaviour, and the full UI.
Streamlit AppTest	—	Renders every page and simulates clicks without a browser.
python-dotenv	≥ 1.0	Environment configuration (<code>.env</code>).

Operational Data — Two Kaggle Datasets

RELAY runs on real, public data rather than hand-written demo records. Two datasets were chosen because no single free dataset covers procurement, logistics and inventory together — each fills exactly the gap the other leaves.

Dataset	What it is	Size	Feeds
SCMS Delivery History	USAID Supply Chain Management System — real medicine and diagnostic-kit deliveries, 2006–2015, 43 countries, 73 vendors.	10,324 line items	Purchase orders, customer orders, shipments, suppliers, carriers, freight cost.
Inventory Management E-Grocery	Warehouse stock snapshot across 5 Indonesian distribution centres.	1,000 SKUs	Warehouses, products, suppliers, live stock levels, reorder points, safety stock.

Turning a finished history into a live snapshot

The SCMS dataset is a closed history — every order in it was, eventually, delivered. To give RELAY a live operational state, the importer (`data/kaggle_import.py`) picks a single date inside that history and treats it as “today”:

- An order issued before that date and delivered before it becomes **history** — its real, recorded lateness is kept.
- An order issued before it but not yet delivered becomes **live**: marked `DELAYED` if already overdue, `IN_TRANSIT` if due soon, otherwise `CREATED`.
- An order issued after that date is **future** and is excluded entirely.

Every date is then shifted forward by a fixed number of days so the chosen date lands on today. Nothing about a live record's eventual, real-world outcome is written into it — the agent only ever sees what would genuinely have been known at that point in time.

What is not in either dataset

A small number of fields — expedite lead-time factors, customer SLA hours by tier, product criticality for medicines — are not recorded by either source. Each one is either derived from a named, documented rule or left empty; none are invented. The full list is in `data/DATA_SOURCES.md`.

Database Design

The schema (database/schema.sql) has 17 tables split into three groups: master data, transactional records, and the agent's own governance trail.

Group	Tables	Purpose
Master data	customers, products, warehouses, suppliers, carriers, routes	The stable entities everything else references.
Transactions	inventory, orders, order_items, purchase_orders, purchase_order_items, shipments	The live operational state the agent reads and, on approval, writes.
Exception & governance	exceptions, resolution_options, actions, audit_logs, data_sources	What the agent found, what it proposed, what was executed, and a full immutable trace of every step.

Design principles

- Referential integrity is enforced with foreign keys and CHECK constraints on every status field (e.g. a shipment's status can only be one of 8 defined values) — invalid states are rejected by the database itself, not just application code.
- Every write the agent can make goes through parameterised SQL — no string-built queries — closing off SQL injection as an attack surface.
- resolution_options.parameters is a JSONB column, so each resolution type can carry its own shape of arguments without new columns per action type.
- Indexes are placed on every foreign key and status column that the UI or agent filters by (orders.customer_id, shipments.status, exceptions.category, and 8 more).

The Agentic Workflow — Seven Stages

Every exception, regardless of type, moves through the same seven-stage lifecycle. This is the core idea of the project: turn an ad-hoc human investigation into a repeatable, auditable process, without removing the human from the decision.

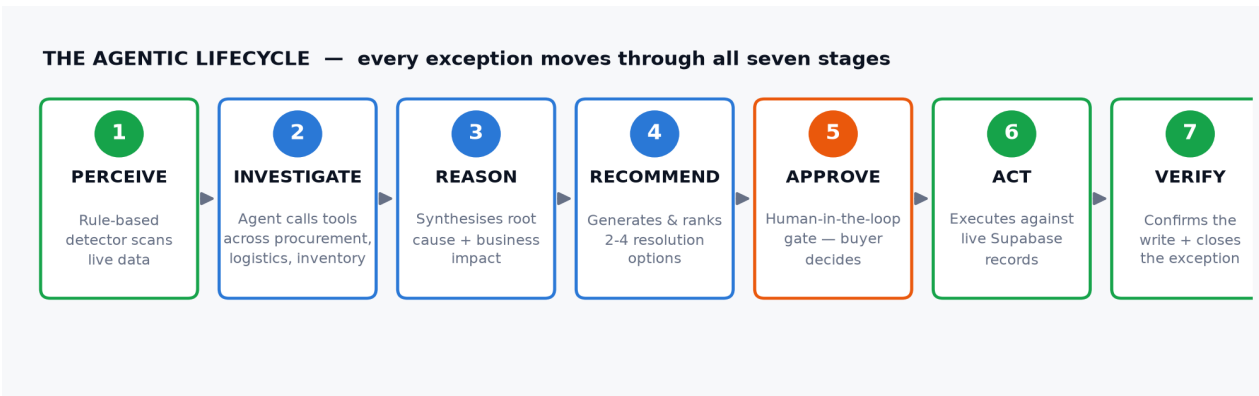


Figure 2 — The seven-stage agentic lifecycle every exception passes through.

1 • Perceive

A deterministic rule scans live data every time it runs — not the LLM. Four rules currently run: shipment delays, purchase-order delays, inventory shortages (stock at or below safety level), and supplier capacity bottlenecks (utilisation $\geq 95\%$). This keeps detection explainable and reproducible.

2 • Investigate

A LangGraph state machine repeatedly asks the LLM (or, if the API is unavailable or rate-limited, a deterministic reasoner with the same decision tree) which of 16 tools to call next, given what has been found so far — up to 6 steps. Each tool call is a real, parameterised SQL query.

3 • Reason

Once enough evidence is gathered, the agent synthesises a root cause and a multi-dimensional business impact (procurement, logistics, inventory, customer, and an overall stockout-risk rating).

4 • Recommend

The agent generates 2–4 candidate resolutions (e.g. inventory transfer, carrier change, supplier switch, expedite, or a new replenishment purchase order), each with an estimated cost, time-to-effect, operational risk and confidence score — then ranks them and picks one to recommend.

5 • Approve

Execution is strictly blocked until a human — a buyer-role user — reviews the evidence and the recommended option, then approves, rejects, or picks an alternative. This gate is enforced in code (`services/resolution_service.py`), not just in the UI: even a direct call to execute a resolution raises a `PermissionError` unless the exception's status is `APPROVED`.

6 • Act

Only after approval does a typed execution tool run the real database write — e.g. moving stock between warehouses, reassigning a shipment's carrier, or raising a replenishment purchase order — inside a single transaction.

7 • Verify

The execution result is confirmed against the database, the exception is marked RESOLVED, and every input, output and decision along the way has already been written to the audit_logs table.

Exception Detection & Resolution Options

Four detection rules, four exception types:

Exception type	Detection rule	Note
Shipment delay	Status is DELAYED, or past its expected delivery date and not yet delivered.	Financial exposure valued at 20% of the goods on board.
Purchase order delay	Status is DELAYED, or past expected delivery and not yet received.	Signals a procurement-side risk to downstream stock.
Inventory shortage	Available stock is at or below the safety-stock threshold.	The most common exception in the current data — 198 of 207.
Supplier capacity bottleneck	A supplier's utilised capacity is 95% or more of its daily maximum.	Flags a systemic risk before it causes a delay.

Six resolution action types

Every resolution option resolves to one of six concrete, executable actions — never a vague suggestion. Each is grounded against real IDs before it can be approved; a proposed action that references a product, warehouse or supplier that does not exist is rejected before it ever reaches the approval screen.

INVENTORY_TRANSFER	Move stock from a warehouse with surplus to the one that is short.
CARRIER_CHANGE	Reassign a delayed shipment to a more reliable carrier / shipping mode.
SHIPMENT_REROUTE	Move a shipment onto an alternative route.
SUPPLIER_SWITCH	Reassign a purchase order to a qualified alternative supplier.
EXPEDITE_PO	Apply priority handling and an expedite cost multiplier to an existing order.
REPLENISHMENT_PO	Raise a new purchase order to the product's usual supplier, sized to its reorder point + safety stock.

Application Modules

Buyer workspace

- **Overview** — KPI tiles (in flight, delayed, open POs, stock at risk, exceptions, awaiting approval), the highest-severity exception queue, and where exceptions come from by category.
- **Exceptions** — the full seven-stage workspace: run an investigation, generate resolutions, review the evidence, approve or reject.
- **Approvals** — every exception waiting on a human decision, with the agent's recommended option, its cost, time-to-effect, risk and confidence.
- **Shipments, Purchase orders, Inventory, Suppliers** — full operational views with search, status filters, and (for purchase orders) a form to raise a new one.
- **Audit trail** — every agent step and human decision ever recorded, filterable by exception and step type.

Supplier workspace

- **Overview** — orders awaiting confirmation, delayed shipments, exposure on the shared record.
- **Purchase orders** — confirm or review incoming orders issued by buyers.
- **Shipments** — outbound movement tracking.
- **Products** — the supplier's own catalogue, editable, priced at cost.
- **Performance** — on-time delivery rate and reliability, computed from the same records the buyer sees.

Both workspaces share one design system (`app/theme.py`, `app/ui.py`) so that badges, tables, KPI tiles and charts read identically across roles, and one cached data-access layer (`app/data.py`, 20-second TTL) so the UI never issues the same query twice within a page render.

Authentication, Roles & Governance

Sign-up and sign-in run through Supabase Auth. A signed-up user picks a role — BUYER or SUPPLIER — which the application layer uses to route them to the correct workspace and to gate what they can do: only a buyer can approve or reject a resolution; a supplier can read the same exception and evidence but has no execute control. A profile page lets a signed-in user update contact details, change their email (which goes through Supabase's own confirmation flow) and change their password (which requires re-entering the current one).

Governance is enforced twice: once in the UI (a supplier session never renders an approve button), and once in the service layer itself — `services/resolution_service.py` raises a `PermissionError` if execution is attempted on an exception that has not been marked APPROVED by a human, regardless of which code path calls it.

Testing Strategy

60 automated tests across four files, each targeting a different layer:

File	Tests	What it covers
tests/test_kaggle_import.py	9	The data importer — number parsing, referential integrity, no future-dated leaks, no invented values.
tests/test_day1.py	8	The live database: schema, referential integrity, the replayed snapshot, and that every delay/shortage has a matching exception.
tests/test_day2.py	6	The agent against real data: tool execution, investigation, and that a real shortage produces a grounded, executable replenishment option.
tests/test_frontend.py	37	Every page, for both roles, rendered through Streamlit's own AppTest — routing, forms, the approval gate, sign-out, and the profile flows.

Tests that exercise the agent stub out every database write, so running the suite never mutates real data. The data-layer tests are read-only by design — they skip rather than reseed if the database is empty.

Results — What the System Detected

On the current data load (SCMS replayed to 5 July 2015, shifted to today), the detector found 207 exceptions:

Exception type	Count	Source
Inventory shortage	198	Products at or below safety stock (E-Grocery)
Shipment delay	8	Shipments past their scheduled delivery (SCMS)
Supplier capacity bottleneck	1	A vendor at ≥95% daily capacity (SCMS)

88.3% on-time delivery	\$761K active exposure	6,786 shipments	2,803 purchase orders
--	--	---------------------------------------	---

Reliability, lead time and freight-cost figures shown throughout the app are computed from the datasets' real delivery history — not assumed defaults.

Challenges & Engineering Decisions

Replaying a finished history as a live system

SCMS ends in 2015 with every order delivered. Solved with the as-of-date replay described in Section 5 — the only way to get a live operational state without inventing data.

Two unrelated networks, one schema

The two datasets share no supplier, product or warehouse. Rather than force an artificial link, they are kept distinct (SCMS-... vs EG-... record IDs) and simply run through the same detection and agent pipeline — each covers what the other cannot.

LLM rate limits

The free-tier Groq API allows 8,000 tokens/minute. Prompts are compacted (long lists and strings truncated) before being sent, and every LLM call has a deterministic fallback so the agent keeps working — just less fluently — if the API is rate-limited or unavailable.

Never inventing an ID

Early iterations let the agent fall back to hardcoded placeholder IDs (a specific warehouse, a specific supplier) when it couldn't resolve a real one. On real data those IDs don't exist. Every fallback was replaced with either a real lookup or a clear, explicit error — the agent now fails loudly rather than acting on a fabricated record.

UI at real-data scale

Demo data had a handful of rows per table; the Kaggle load has thousands. Every list page was rebuilt with server-side filtering, search, and a capped render (150 rows) so the browser stays responsive.

Limitations & Future Scope

Current limitations

- The two datasets are not cross-linked — a shipment delay in SCMS cannot trigger a stock check in the grocery data, since they describe different networks.
- No route or distance data exists in either source, so the agent cannot propose a reroute across cities.
- The E-Grocery dataset appears to be synthetically generated (patterned product names); it is used as published since it is publicly available and clearly documented as such.

Future scope

- Bring in a dataset that links shipments to stock levels within one network, closing the delay → shortage → customer-risk chain end to end.
- Add a forecasting agent that predicts a shortage before it crosses the safety-stock line, not just after.
- Extend the audit trail into a compliance export (PDF/CSV) for procurement teams.
- Add role-based analytics dashboards comparing supplier and carrier performance over time.

Conclusion

RELAY demonstrates a complete, working pattern for agentic AI in an operational setting: a deterministic trigger, an LLM-driven investigation grounded in real tool calls, a ranked set of concrete options, a hard human approval gate, and an immutable audit trail — all running against real, verifiable data rather than a scripted demo. The result is a system that does the tedious cross-referencing work automatically while leaving every consequential decision to a person, with a complete record of why.

Report generated for RELAY — Agentic Supply-Chain Exception Resolution System